DBMS — Relational Model

Complete One-Read Study Notes with Color-Coded Concepts, Examples & Numericals

Unit 1 & 2 · Compiled for Deekshant · MITS Gwalior

Colour Code Legend

- **Blue** — Definition / Concept
- **Green** — Example
- **Yellow/Orange** — Numerical / Practice Problem
- **Red** — Important Property / Rule (exam-critical)
- **Purple** — Extra Note / Tip
- **Teal** — Worked Solution

1. Relational Model — The Basics

DEFINITION: The **Relational Model** is a data model that represents a database as a collection of **relations (tables)**, where each relation has **rows (tuples)** and **columns (attributes)**. Proposed by **E. F. Codd in 1970**. It is an **Implementation/Representational model** — meaning it's how the database actually gets built (unlike ER model, which is only conceptual).

REAL-WORLD EXAMPLES: MySQL, Oracle, PostgreSQL, SQL Server — all of these are RDBMS (Relational Database Management Systems) built on this model.

Core Terminologies (memorize with examples!)

Term	Meaning	Example
Relation	A table with a unique name — made of rows & columns	`STUDENT(Roll_no, Name, Branch)`
Tuple	One single row of a relation (one record)	`(101, "Amit", "CSE")`
Attribute	One column — a property of the entity	`RollNo, Name`
Domain	Set of all legal/possible values an attribute can take	Domain(Age) = {1–120}
Degree	Number of attributes (columns) in a relation	4 columns → Degree = 4
Cardinality	Number of tuples (rows) in a relation	50 rows → Cardinality = 50
Database Intension	The schema/structure of the DB (not the data itself) — also called "Database Schema". Rarely changes.	Table structure definitions

TIP TO REMEMBER: Degree = columns (think "D for Down the top row"), Cardinality = rows (think "how many Cards/records you have").

QUICK NUMERICAL: If relation R(A1, A2, A3, A4, A5) has 5 attributes and 40 rows of data → **Degree = 5**, **Cardinality = 40**. Also, mathematically: `r(R) ⊆ D1 × D2 × D3 × D4 × D5` — meaning every tuple is one combination picked from the Cartesian product of all attribute domains.

2. Keys in DBMS

DEFINITION: A **Key** is an attribute or a combination of attributes that **uniquely identifies** each tuple (row) in a table.

Super Key (all uniquely-identifying sets)
Candidate Key (minimal super keys)
Primary Key (the ONE chosen)

a) Super Key

DEFINITION: Any combination of attributes that uniquely identifies a record. Can have **extra/redundant** attributes.

EXAMPLE: Student(roll, name, sex, age, address, class, section) — Super Keys include:

- (roll, name, sex, age, address, class, section)
- (class, section, roll)
- (class, section, roll, sex)
- (name, address)

b) Candidate Key (Minimal Super Key)

DEFINITION: A super key with **no unnecessary/extra attributes** — the smallest possible set that still uniquely identifies each tuple.

EXAMPLE (same Student table): Candidate Keys = (class, section, roll) and (name, address) — nothing more can be removed from either set.

PROPERTIES (exam favourite!):

- All attributes are necessary AND sufficient to identify each tuple
- Removing any attribute breaks uniqueness
- Value must always be unique & can never be NULL
- A relation can have **multiple** candidate keys
- Attributes appearing in any candidate key = **Prime Attributes**

c) Primary Key

DEFINITION: The **one candidate key** that the database designer selects as the main/official way to identify tuples.

PROPERTIES:

- Value can **never** be NULL
- Value must always be unique
- Value can **never be updated/changed**
- Must be assigned at the time of inserting a record
- A relation is allowed **only ONE** primary key

d) Alternate Key

DEFINITION: Candidate keys that were **not chosen** as the primary key — i.e., the "leftover" candidate keys.

e) Foreign Key

DEFINITION: Attribute 'X' is a foreign key to attribute 'Y' when X's values depend on / must come from Y's values. The table containing Y = **Referenced (master) relation**. The table containing X = **Referencing (foreign) relation**.

WORKED EXAMPLE:

Department (Referenced) — PK: dept_no	
dept_no	dept_name
D01	Computer Science
D02	Mathematics
D03	Physics

Teacher (Referencing) — PK: t_no, FK: dept_no → Department(dept_no)			
t_no	t_name	t_age	***dept_no***

T101	Ravi	35	D01
T102	Anita	29	D02
T103	Suresh	42	D01

PROPERTIES OF FOREIGN KEY:

- References the primary key of another (or same) table
- Can take **only** values present in the referenced PK, or NULL
- Can have a different name than the PK it references
- **Can be NULL** (unlike primary key)
- **No uniqueness restriction** — usually repeats (not unique)

3. Relational Constraints

DEFINITION: Relational (Integrity) constraints are **rules** imposed on data to maintain correctness and quality of information. 4 types: **Domain, Key, Entity Integrity, Referential Integrity**.

1. Domain Constraint

RULE: Value of an attribute must be an **atomic** value from its defined domain. No composite or multi-valued entries allowed.

VIOLATION EXAMPLE: Age column has value `"A"` instead of a number → violates domain constraint. Similarly, a Phone_no cell storing **two numbers** (8988234351, 9190156244) in one field violates atomicity.

2. Key Constraint

RULE: All values of the **primary key** must be unique — no two tuples can share the same PK value ("uniqueness constraint").

VIOLATION EXAMPLE: Two rows both have STU_ID = `S001` → violates key constraint.

3. Entity Integrity Constraint

RULE: No attribute belonging to the **primary key** can ever be NULL.

VIOLATION EXAMPLE: STU_ID = `NULL` for student "Rahul" → violates entity integrity, since STU_ID is the PK.

4. Referential Integrity Constraint

RULE: Every foreign key value must either (a) match a value existing in the referenced relation's primary key, or (b) be NULL.

EXAMPLE: Teacher.dept_no can only contain D01/D02/D03 (values that exist in Department.dept_no) or NULL — it cannot contain "D99" if that department doesn't exist.

⚡ **Quick Recall Table**

Constraint	Applies to	Core Rule
Domain	Any attribute	Value must be atomic & in-domain
Key	Primary Key	Must be unique
Entity Integrity	Primary Key	Cannot be NULL
Referential Integrity	Foreign Key	Must exist in referenced PK, or be NULL

4. E-R to Relational Mapping (7 Rules)

WHY: Before implementing a database, we must convert the conceptual **E-R diagram** into a **relational model** (actual tables). There are **7 standard rules**.

Rule 1: Strong Entity → Relation

Every strong entity becomes one table. Simple attributes → columns. Composite attributes → break into simplest parts. Multivalued attributes → NOT included directly (handled in Rule 6). Prime attribute (PK in ER) → PK in table.

EXAMPLE: Student entity with SName(FName, LName), Roll-No (key), Marks, Mobile_Number(multivalued) →

Student			
Roll-No	Marks	FName	LName

(Mobile_Number is handled separately — see Rule 6)

Rule 2: Weak Entity → Relation

Weak entity has no PK of its own — only a **partial key**. Create its table with all its attributes + add the **owner entity's PK** as a foreign key. New PK = **(Owner's PK + Partial Key)**.

EXAMPLE: Employee(E-No is PK) —Have→ Dependent(DName is partial key, Age, Relation)

Employee	
E_no	EName

Dependent			
E-No	**DName**	Age	Relation

PK of Dependent = **E-No + DName** | FK of Dependent = **E-No**

Rule 3: 1-to-1 Relationship

Add the PK of one entity as a FK into the other entity's table (usually into the side with **partial participation**). If BOTH sides have **total participation**, you can simply **merge both tables into one**.

EXAMPLE: Employee —Manages(1:1)→ Department, with attribute Start-Date on relationship → add to total-participation side:

Department (final)			
D-No	DName	***Manager-No***	Start-Date

Rule 4: 1-to-Many (1:N) Relationship

Put the FK on the **"Many" side**. Any attribute of the relationship also goes to the Many side.

EXAMPLE: Department(1) —Works-for→ Employee(N), attribute Start-Date on relationship:

Employee (final)			
E-No	EName	***D-No***	Start-Date

Rule 5: Many-to-Many (M:N) Relationship

Cannot put FK on either side. Must create a **brand-new junction/bridge table** holding **both PKs as FKs**. New table's PK = **combination of both FKs**. Relationship's own attributes go here too.

EXAMPLE: Employee(M) —Works-for→ Project(N), attribute Hours:

Works-for (new junction table)		
E-No	***P-No***	Hours

Rule 6: Multivalued Attribute

Cannot store directly in the entity's table. Create a **new table** with the multivalued attribute + the owner entity's PK as FK. New PK = combination of both.

EXAMPLE: Student has multivalued Mobile_Number:

Mobile_Number (new table)	
S-No	**Mobile_Number**

PK = S-No + Mobile_Number | FK = S-No

Rule 7: M-ary Relationship (>2 entities in one relationship)

Create a separate table for the relationship itself. Add PKs of **all participating entities** as FKs. New PK = combination of all those FKs. Relationship's own attributes also added here.

EXAMPLE: Entities E1,E2,E3,E4 with attributes A1,A2,A3,A4 join in relationship R:

Table for Relationship R			
A1	***A2***	***A3***	***A4***

PK = A1+A2+A3+A4 | FK = A1, A2, A3, A4 (each referencing its own entity)

5. Practice Numericals (Worked Out)

GOLDEN RULE FOR THESE PROBLEMS: Merge an entity with a 1:1 or 1:N relationship into ONE table when possible (put relationship on the "1" side or the side that already has the FK direction). M:N relationships and M-ary relationships ALWAYS need their own separate table. Weak entities merge with their identifying relationship.

PROBLEM 1: M —R1→ P —R2→ N (chain of two binary relationships, arrows show directional/1-side). M has attributes M1(key),M2,M3. P has P1(key),P2. N has N1(key),N2.

SOLUTION — Minimum 3 tables:

- MR1 (M1, M2, M3, P1) — merges M with relationship R1
- P (P1, P2)
- NR2 (P1, N1, N2) — merges N with relationship R2

PROBLEM 2: A links to B via R1, A links to C via R2, B links to C via R3 (a triangle of relationships). A has a1,a2. B has b1,b2. C has c1,c2.

SOLUTION — Minimum 4 tables:

- AR1R2 (a1, a2, b1, c1)
- B (b1, b2)
- C (c1, c2)
- R3 (b1, c1)

PROBLEM 3: A—R1—B, A—R2—C, B—R4—D, C—R3—D, and a cross-relationship R5 between B and C. A(a1,a2), B(b1,b2), C(c1,c2), D(d1,d2).

SOLUTION — Minimum 5 tables:

- BR1R4R5 (b1, b2, a1, c1, d1)
- A (a1, a2)
- R2 (a1, c1)
- CR3 (c1, c2, d1)
- D (d1, d2)

PROBLEM 4: E1—R1—E2—R2—E3 (chain). E1(a1,a2), E2(b1,b2,b3), E3(c1,c2).

SOLUTION — Minimum 3 tables:

- E1 (a1, a2)
- E2R1R2 (b1, b2, a1, c1, b3)
- E3 (c1, c2)

PROBLEM 5 (Classic Bank Schema): Account(Ac_no, Balance) —Account_branch(1:1 total on Account side)→ Branch(b_name, b_city, Assets). Account —Depositor(M:N)→ Customer(C_name, C_street, C_city). Customer —Borrower(M:N)→ Loan(L_no, Amt). Loan —Loan_branch(1:1)→ Branch.

SOLUTION — Minimum 6 tables:

- Account (Ac_no, Balance, b_name) — merged with Account_branch since Account has total participation
- Branch (b_name, b_city, Assets)
- Loan (L_no, Amt, b_name) — merged with Loan_branch
- Borrower (C_name, L_no) — M:N needs its own table
- Customer (C_name, C_street, C_city)
- Depositor (C_name, Ac_no) — M:N needs its own table

One-Minute Full Recap

- **Relational Model:** tables = relations, rows = tuples, columns = attributes. By E. F. Codd, 1970.
- **Degree** = #columns, **Cardinality** = #rows.
- **Keys ladder:** Super Key ⊇ Candidate Key ⊇ Primary Key. Unused candidate keys = Alternate Keys. Foreign Key links tables.
- **4 Constraints:** Domain (atomic values) · Key (PK unique) · Entity Integrity (PK never NULL) · Referential Integrity (FK matches PK or NULL).
- **ER→Relational, 7 rules:** Strong entity=table · Weak entity+partial key+owner PK · 1:1 merge or FK on either side · 1:N FK on Many side · M:N new junction table · Multivalued attribute new table · M-ary new table with all PKs.